

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
HIMALAYA COLLEGE OF ENGINEERING**

[CODE: ENCT 354]



**A MID-TERM PROGRESS REPORT
ON
AN INTELLIGENT ADAPTIVE MULTI-VENDOR ELECTRONICS
MARKETPLACE WEB APPLICATION**

Submitted By

Aastha Mainali (HCE080BCT001)

Aayushma Ghale (HCE080BCT003)

Anjila Pandey (HCE080BCT007)

Anshu Regmi (HCE080BCT009)

A Mid-Term Progress Report Submitted to Department of Electronics and Computer
Engineering in Partial Fulfillment of the Requirement for Bachelor's Degree in
Computer Engineering

Department of Electronics and Computer Engineering

Lalitpur, Nepal

July 27, 2026

AN INTELLIGENT ADAPTIVE MULTI-VENDOR ELECTRONICS MARKETPLACE WEB APPLICATION

Submitted By

Aastha Mainali (HCE080BCT001)

Aayushma Ghale (HCE080BCT003)

Anjila Pandey (HCE080BCT007)

Anshu Regmi (HCE080BCT009)

PROJECT SUPERVISOR

Er. Isan Adhikari

A project progress report submitted in partial fulfillment of the
requirements for the
Bachelor's Degree in Computer Engineering

HIMALAYA COLLEGE OF ENGINEERING

Department of Electronics and Computer Engineering

Tribhuvan University

Lalitpur, Nepal

July 27, 2026

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Himalaya College of Engineering, for the opportunity and resources to undertake this project. We are especially thankful to our project supervisor, **Er. Isan Adhikari**, for his continuous guidance, valuable feedback, and encouragement throughout the development of this project. We also extend our appreciation to the Head of Department, Deputy Head, and Project Coordinator for their support and coordination, and to everyone who has directly or indirectly contributed to this work.

ABSTRACT

This report presents an intelligent multi-vendor electronics marketplace that enables customers to discover, compare, and purchase smartphones and laptops from multiple verified sellers on a single platform. It addresses a key limitation of existing e-commerce platforms: product discovery that relies on exact keyword matching and offers no personalized assistance to users unfamiliar with precise product names and specifications. The platform supports three roles, customer, seller, and administrator, each with dedicated functionality and role-based access control.

Two core intelligent features have been implemented so far, with a third planned for the remaining phase: a content-based recommendation engine that suggests similar devices from the currently viewed product without requiring purchase history, and a semantic keyword search interface for locating products by category, brand, and specification. An OpenCV-based visual search capability, allowing customers to upload a product photograph and retrieve visually matching catalog items, is planned for the remaining development phase. The platform is built using React.js, Django REST Framework, and MySQL, with Khalti integrated for payment processing. The recommendation engine, built using weighted cosine similarity over TF-IDF text vectors, one-hot categorical vectors, and min-max scaled specification vectors, has been implemented, tuned, and independently validated against a random-baseline recommender. The outcome is expected to be a fully functional, intelligent marketplace that supports informed purchasing decisions across all customer demographics, while giving sellers and administrators structured tools for product and platform management.

Keywords: *content-based filtering, cosine similarity, e-commerce, marketplace, recommendation system, semantic search*

TABLE OF CONTENTS

ACKNOWLEDGMENTS	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Background Information	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope and Applications	3
2 LITERATURE REVIEW	4
2.1 E-Commerce Platforms and System Architecture	4
2.2 AI-Based Recommendation Systems and Behavior Tracking	5
2.3 Product Search and Information Retrieval	5
2.4 Identified Research Gaps	6
3 REQUIREMENT ANALYSIS AND SYSTEM DESIGN	7
3.1 Requirement Analysis	7
3.1.1 Functional Requirements	7
3.1.2 Non-Functional Requirements	9
3.2 Feasibility Analysis	9
3.3 System Design	10
3.3.1 ER Diagram	10
3.3.2 Data Flow Diagram	12
3.4 Activity Diagram	14

4	METHODOLOGY	16
4.1	Basic System Architecture	16
4.1.1	System Flowcharts	17
4.1.2	Algorithm: Cosine Similarity for Product Recommendation . . .	18
4.2	Agile SCRUM Model	23
4.3	Dataset Description	24
4.4	Tools Used	25
5	RESULTS AND ANALYSIS	26
5.1	Work Completed	26
5.1.1	Backend Development (Django REST Framework)	26
5.1.2	Recommendation Engine Development and Evaluation	27
5.1.3	Frontend Development (React with Tailwind CSS)	30
5.1.4	Version Control and Team Collaboration	30
5.1.5	Verification	30
5.1.6	Validation	31
5.2	Work Remaining	31
6	CONCLUSION	33
	REFERENCES	34
	APPENDICES	36
6.1	APPENDIX A: Project Timeline	36
6.2	APPENDIX B: Technical Specifications	37
6.2.1	Software Requirements	37
6.2.2	Hardware Requirements	37
6.3	APPENDIX C: Recommendation Engine Terminal Output	38
6.4	APPENDIX D: User Interface Screenshots	39

LIST OF FIGURES

3.1	Use Case Diagram of the Smartphone and Laptop Marketplace	8
3.2	Entity Relationship Diagram	11
3.3	DFD Level 0	12
3.4	DFD Level 1	13
3.5	Activity Diagram - Login, Role Assignment & Product Discovery . . .	14
3.6	Activity Diagram - Cart & Order placement & Payment	15
4.1	System Architecture Diagram	16
4.2	Recommendation Engine Flow Diagram	17
4.3	Agile Model Diagram	23
5.1	Weight Sensitivity Analysis – Text Weight	27
5.2	Weight Sensitivity Analysis – Category Weight	28
5.3	Weight Sensitivity Analysis – Spec Weight	28
5.4	Recommendation Quality: Model vs Random Baseline	29
5.5	Recall@5 versus its Theoretical Ceiling	29
6.1	Project Timeline Gantt Chart	36
6.2	Terminal Output: Top-5 Recommendation Demonstration	38
6.3	Terminal Output: Top-5 Recommendation Demonstration	38
6.4	Registration Interface	39
6.5	Sign In Interface	39
6.6	User Preference Selection Interface	40
6.7	Seller Onboarding Interface	40
6.8	Seller Dashboard Interface	41
6.9	Featured Products Interface	41
6.10	Product Detail Interface	42

LIST OF TABLES

4.1	Development Tools and Technologies	25
6.1	Software Requirements of the Proposed System	37

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CBF	Content-Based Filtering
CF	Collaborative Filtering
CRUD	Create, Read, Update, Delete
DRF	Django REST Framework
HTTP	HyperText Transfer Protocol
MRR	Mean Reciprocal Rank
MVC	Model-View-Controller
MySQL	My Structured Query Language
NDCG	Normalized Discounted Cumulative Gain
OpenCV	Open Source Computer Vision Library
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single-Page Application
TF-IDF	Term Frequency–Inverse Document Frequency

CHAPTER 1:

INTRODUCTION

1.1 Background Information

Retail has shifted decisively from brick-and-mortar stores to digital e-commerce, with smartphones and laptops among the highest-revenue categories. In developing markets such as Nepal, platforms like Daraz show strong consumer demand for computing devices, but they frequently rely on generic keyword search and rigid filtering, offering little personalization. When shopping for high-involvement electronics, users must manually parse dense, rapidly evolving specifications such as RAM, storage, and processor generation across volatile price points.

Content-based filtering driven by cosine similarity can map relationships between devices by converting technical specifications and descriptions into comparable vectors, while computer vision enables visual search, letting users find matching devices from a photograph rather than text alone.

This project addresses these gaps with an adaptive multi-vendor marketplace dedicated to smartphones and laptops. Built on a React.js frontend, a Django backend, and a MySQL database, the platform manages access control via JSON Web Tokens (JWT) across three user roles: customer, seller, and administrator. Its intelligence layer combines a semantic keyword search engine and a content-based recommendation matrix, with an OpenCV-based visual search pipeline planned for the remaining phase. The recommendation matrix processes item vectors across three dimensions, text descriptions (TF-IDF), categorical attributes (one-hot encoding), and numerical specifications (min-max scaling), with weights optimized via a grid search that was independently validated to avoid relying on a single, potentially fragile search run.

The platform also binds this recommendation matrix to a transactional activity tracker. As a customer browses, the application logs item views, cart updates, and purchase history, blending these signals with the similarity space to regenerate personalized carousels such as a "Featured for You" homepage panel, closing the loop from device discovery to order processing.

1.2 Problem Statement

Existing e-commerce platforms in the Nepali and broader South Asian market face several limitations when handling complex categories like smartphones and laptops:

- **Text-locked discovery:** Users must know exact model names or brands to find items; the absence of semantic matching and visual search restricts discovery for non-technical customers.
- **Static, unpersonalized catalogs:** Most platforms show an identical catalog to every user and do not track browsing or cart behavior, so listings ignore individual budgets and specifications.
- **Biased, uncalibrated recommendations:** Existing carousels rely on simple brand matching, producing brand-lock bias instead of price-sane alternatives across competing brands.
- **Disconnected search and specifications:** Text, category, and numeric parameters are processed in isolation, with no normalization layer combining them into a single similarity space.
- **Weak vendor onboarding:** Platforms lack a structured seller dashboard and admin approval queue, leading to unreliable listings.
- **High friction, high abandonment:** Parsing dense specifications creates cognitive load, resulting in poor discovery and higher cart abandonment.

1.3 Objectives

The specific objectives of this project are:

- To build a secure, full-stack multi-vendor marketplace using React.js and MySQL, with dedicated customer, seller, and administrator interfaces and robust order processing.
- To enhance product discovery using a cosine-similarity recommendation engine driven by real-time browsing behavior, combined with semantic keyword search and an integrated visual search system.

1.4 Scope and Applications

The platform is engineered to optimize the digital retail lifecycle for consumer computing hardware. Key applications include:

- **Specialized e-commerce marketplaces:** A dedicated retail portal for high-involvement technology niches such as independent hardware stores and smart-phone distributors.
- **Localized multi-vendor ecosystems:** A scalable framework for regional tech hubs in Nepal to onboard multiple vendors into one admin-moderated platform.
- **Intelligent comparison portals:** The hybrid specification matching matrix can support an unbiased, third-party electronics evaluation platform.
- **Personalization testing:** The behavioral tracking architecture serves as a blueprint for updating recommendation models from live user interactions.

CHAPTER 2:

LITERATURE REVIEW

This chapter reviews prior work relevant to the proposed system, organized across three domains: e-commerce platform architecture, content-based recommendation systems, and product search and information retrieval.

2.1 E-Commerce Platforms and System Architecture

Electronic commerce is defined as the process of buying, selling, or exchanging products, services, and information via computer networks, with multi-role systems comprising customers, sellers, and administrators identified as a core architectural requirement of modern marketplace platforms [1]. This three-actor model forms the structural basis of the proposed system.

E-commerce platforms are further categorized by business model, distinguishing business-to-consumer (B2C) platforms from marketplace models where third-party sellers manage their own inventory [2]. The proposed system adopts the marketplace model, requiring seller registration and verification, an administrative control panel, and role-based access control.

Separating frontend and backend concerns through RESTful API design enables scalable, loosely coupled systems whose client and server components can evolve independently [3], a principle the proposed system implements by combining a React.js single-page application (SPA) with a Django backend communicating through JSON-based endpoints. RESTful APIs are further shown to provide a standardized, stateless communication model suited to applications serving multiple client types [4], in this case customer, seller, and administrator interfaces from a single backend.

Despite the maturity of platforms such as Amazon and Daraz, most conventional systems present the same catalog to every user, ignoring preference and browsing history. The proposed system addresses this by integrating an adaptive personalization layer directly into its architecture.

2.2 AI-Based Recommendation Systems and Behavior Tracking

Item-to-item collaborative filtering at Amazon demonstrated that recommendations derived from user behavior and purchase history can substantially increase product discovery and sales conversion, establishing much of the foundation for recommendation system research [5]. Recommendation approaches are more broadly classified into collaborative filtering (CF), content-based filtering (CBF), and hybrid methods, where CF relies on user interactions and CBF on product attributes [6].

The proposed system implements content-based filtering using cosine similarity, a deliberate choice suited to the cold-start problem: unlike collaborative filtering, which needs large historical datasets, content-based filtering can generate recommendations from the first interaction, and is effective in sparse data environments [7]. Deep learning approaches to recommendation are powerful but require large datasets and significant compute [8]; for this system, a lightweight content-based model over specialized specifications (TF-IDF, one-hot encoding, min-max scaling) offers a better balance of performance and feasibility.

A recurring concern in offline evaluation of content-based recommenders is the choice of relevance label: deriving it from one of the same features used to build the recommendation vectors risks a circular evaluation that rewards weights for reconstructing an already-known label rather than genuinely improving quality [6]. This concern directly informed the weight-search and validation methodology described in Chapter 4.

Capturing sequential interaction data, such as item views, cart additions, and purchase history, allows a platform to update its similarity computations in real time, tracking shifting user intent within a session [5].

2.3 Product Search and Information Retrieval

Vector-based information retrieval models for matching queries with relevant documents [9] extend naturally to product search using attribute vectors, enabling semantic search that understands contextual phrases instead of forcing rigid keyword matches. Faceted search, allowing multiple filters such as category, brand, and price, improves user experience [10] and is incorporated into the proposed search engine for browsing complex specification layers.

OpenCV and convolutional computer vision techniques for feature extraction and image comparison [12] will be applied in the remaining development phase to build a visual product search pipeline, letting customers discover matches from a photograph rather than text alone. Navigation and labeling structure significantly affect usability [11]; the proposed system integrates semantic search and faceted filtering into a single interface, with visual search to be layered on as the interface matures.

2.4 Identified Research Gaps

- (i) Cold-start recommendation is addressed by content-based methods, but its evaluation often relies on a relevance definition that is circular with respect to the features being weighted, an issue rarely addressed explicitly in smaller-scale academic implementations.
- (ii) Visual search remains underused in smaller e-commerce platforms.
- (iii) Search, filtering, and similarity matching are usually implemented as separate systems rather than a unified pipeline.
- (iv) Real-time behavior tracking is rarely fed back instantly to update local content-based similarity models.
- (v) Laptop- and smartphone-specific specification personalization remains limited in general platforms.

The proposed system integrates all these components into a unified architecture for smartphone and laptop e-commerce.

CHAPTER 3:

REQUIREMENT ANALYSIS AND SYSTEM DESIGN

3.1 Requirement Analysis

3.1.1 Functional Requirements

a) Customer Module:

- Secure registration and login.
- Semantic keyword search with faceted filtering (price, RAM, storage, brand, OS).
- Visual image search by photo upload (*planned for the remaining phase*).
- Real-time behavior tracking (views, browsing sequence, cart activity).
- Personalized homepage and item-detail recommendation carousels.
- Persistent cart and wishlist with stock checking.
- Multi-stage order placement, tracking, and review posting.

b) Seller Module:

- Registration, profile management, and verification.
- Seller dashboard with real-time sales and revenue analytics.
- Product catalog management (listing, editing, images, price in NPR, stock).
- Order fulfillment tracking and status updates.

c) Administrator Module:

- Management of users, products, brands, and categories.
- Admin dashboard for marketplace-wide monitoring.
- Seller verification queue.
- Global order management and product moderation.

3.1.1.1 Use Case Diagram

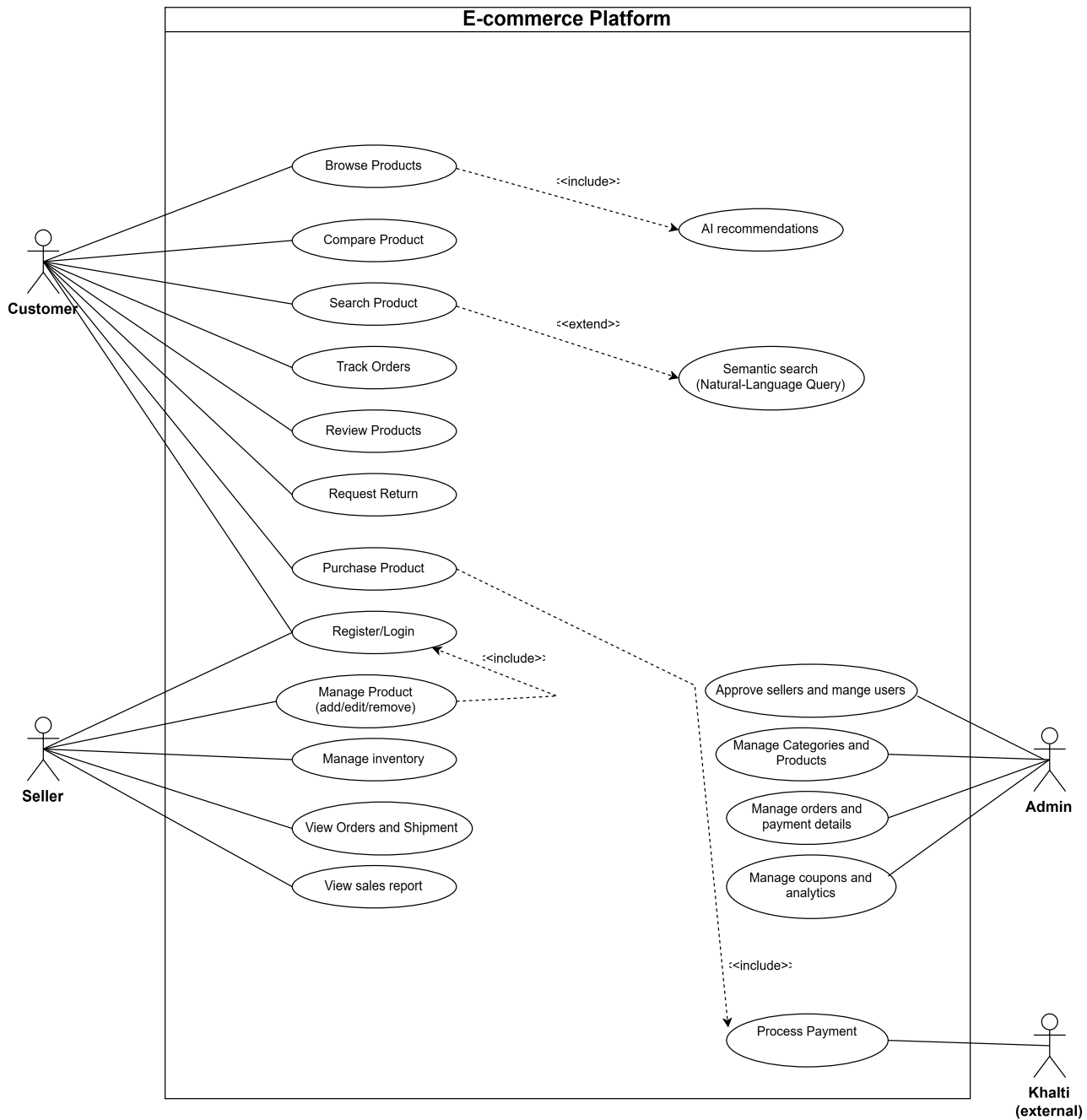


Figure 3.1: Use Case Diagram of the Smartphone and Laptop Marketplace

The diagram isolates how each actor interacts with the system: customers use semantic search, tracking-driven recommendations, and (in the remaining phase) visual search; sellers manage inventory; administrators moderate the marketplace.

- **Customer:** Registers/logs in; browses, searches, and filters products; manages cart/wishlist; triggers behavioral tracking; completes orders.
- **Seller:** Manages product stock, pricing, and specifications; views revenue analyt-

ics; handles order fulfillment.

- **Administrator:** Approves or rejects seller verification; manages categories and brands; audits order logs.

3.1.2 Non-Functional Requirements

- **Performance:** API responses under 2.0 s; similarity/recommendation updates under 3.0 s.
- **Scalability:** Horizontal scaling for growing catalog size and concurrent checkout traffic.
- **Security:** JWT-based sessions, bcrypt-hashed passwords, strict role-based access control (RBAC).
- **Usability:** Intuitive SPA views; semantic queries parsed without prior knowledge of exact model numbers.
- **Reliability:** ACID-compliant transactions; $\geq 99\%$ evaluation uptime.
- **Maintainability:** Modular, separation-of-concerns code; ORM-managed database access for portability.

3.2 Feasibility Analysis

Technical: The React.js, Django REST Framework, and MySQL stack is mature and well documented. Scikit-learn handles TF-IDF, one-hot encoding, and min-max scaling; the remaining phase adds OpenCV for computer vision, with no dependency on heavy external deep-learning infrastructure.

Operational: The platform maps onto standard e-commerce retail workflows in a South Asian market with rising demand for specialized electronics procurement channels.

Economic: All frameworks and storage engines used are open-source; cloud sandboxes, campus infrastructure, and Git are available at no cost, limiting total expenditure to printing and binding.

Schedule: Development follows Agile SCRUM across one academic semester, with sprints isolating schema design, feature-vector tuning, tracking integration, and final UI connections.

3.3 System Design

3.3.1 ER Diagram

The ER diagram shows relationships among the system's primary entities: User, Seller, Product, Brand, Category, Product Specification, Order, Order Item, Cart, Review, and Wishlist. A User places many Orders and may register as a Seller; a Seller lists many Products; a Product belongs to a Brand and a Category; an Order contains many Order Items; a User writes Reviews and saves Wishlists.

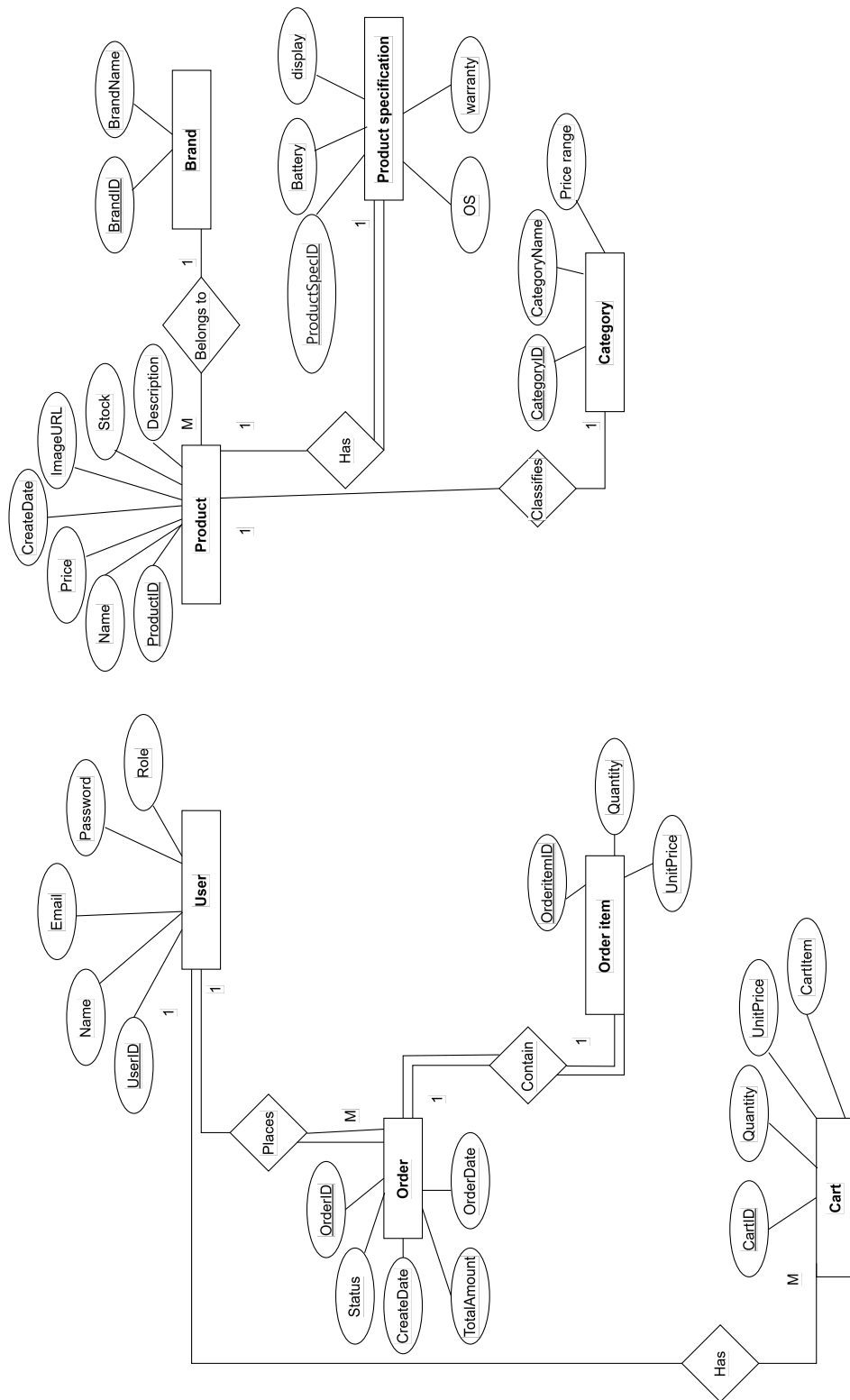


Figure 3.2: Entity Relationship Diagram

3.3.2 Data Flow Diagram

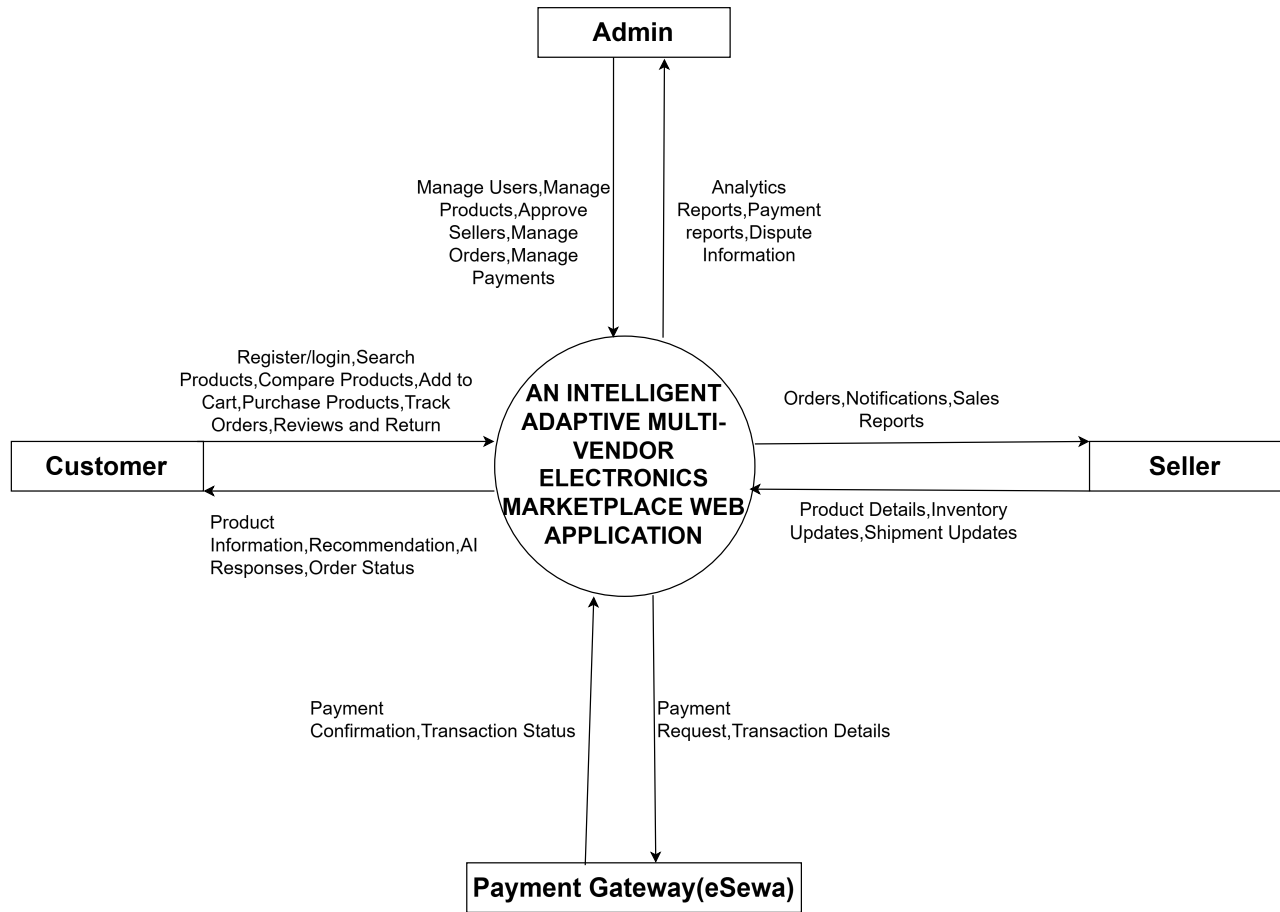


Figure 3.3: DFD Level 0

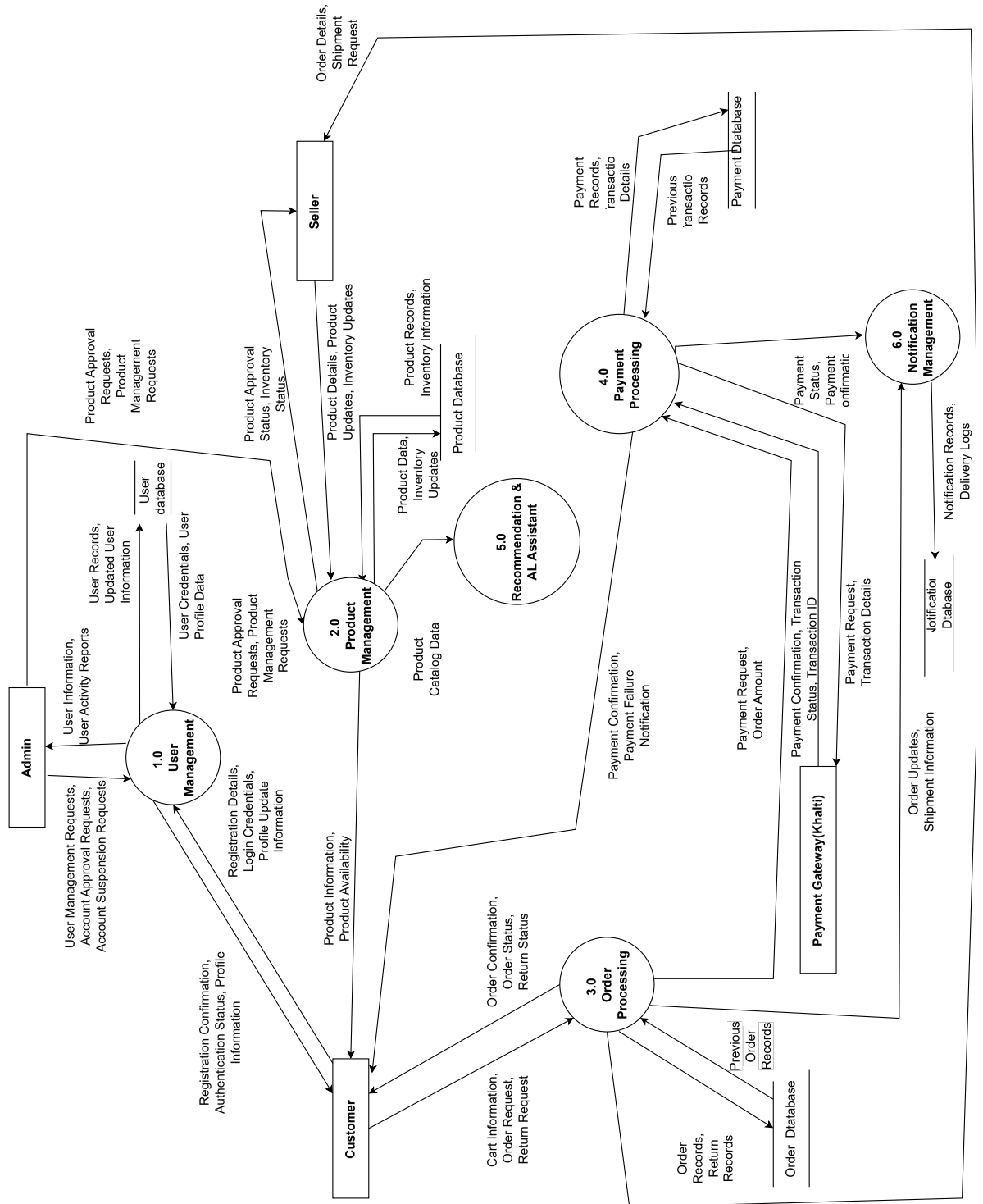


Figure 3.4: DFD Level 1

3.4 Activity Diagram

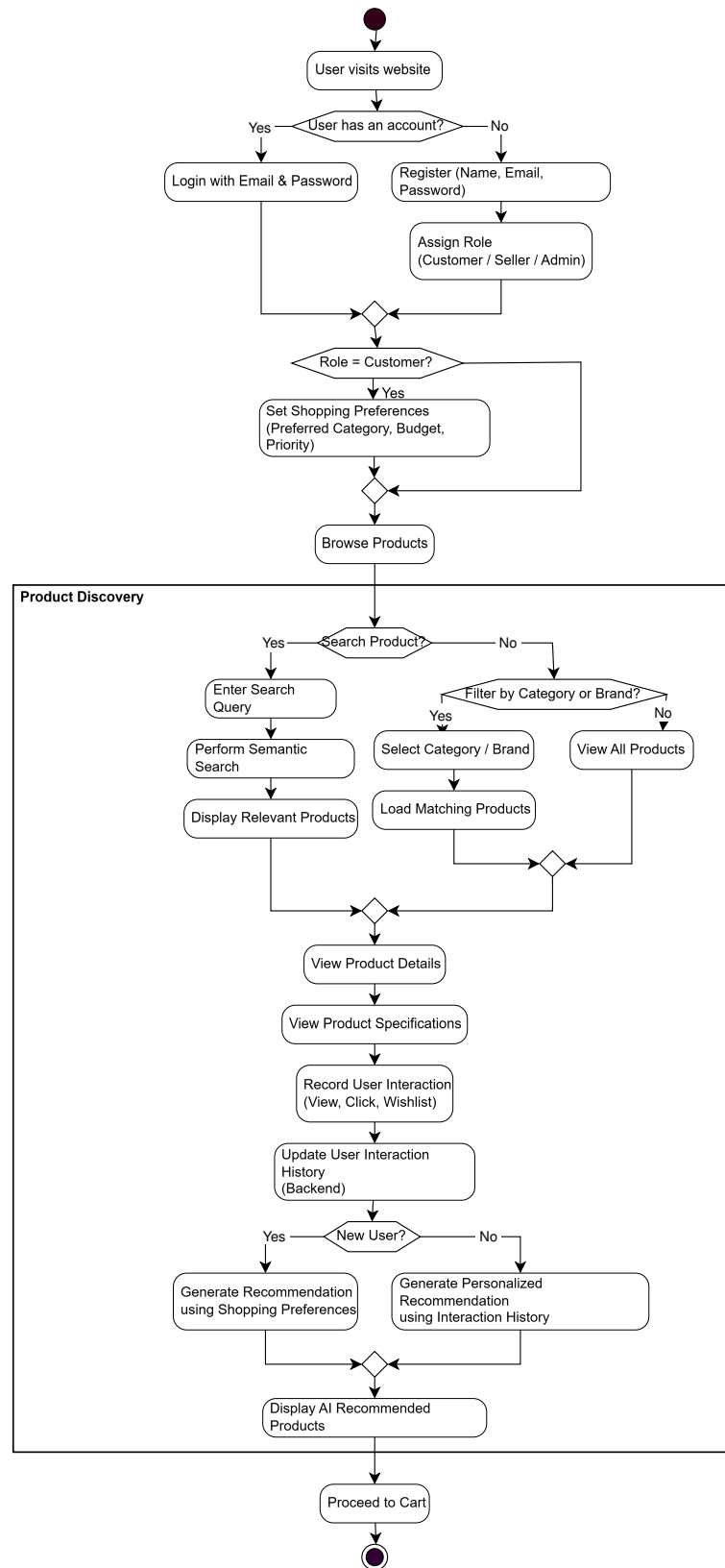


Figure 3.5: Activity Diagram - Login, Role Assignment & Product Discovery

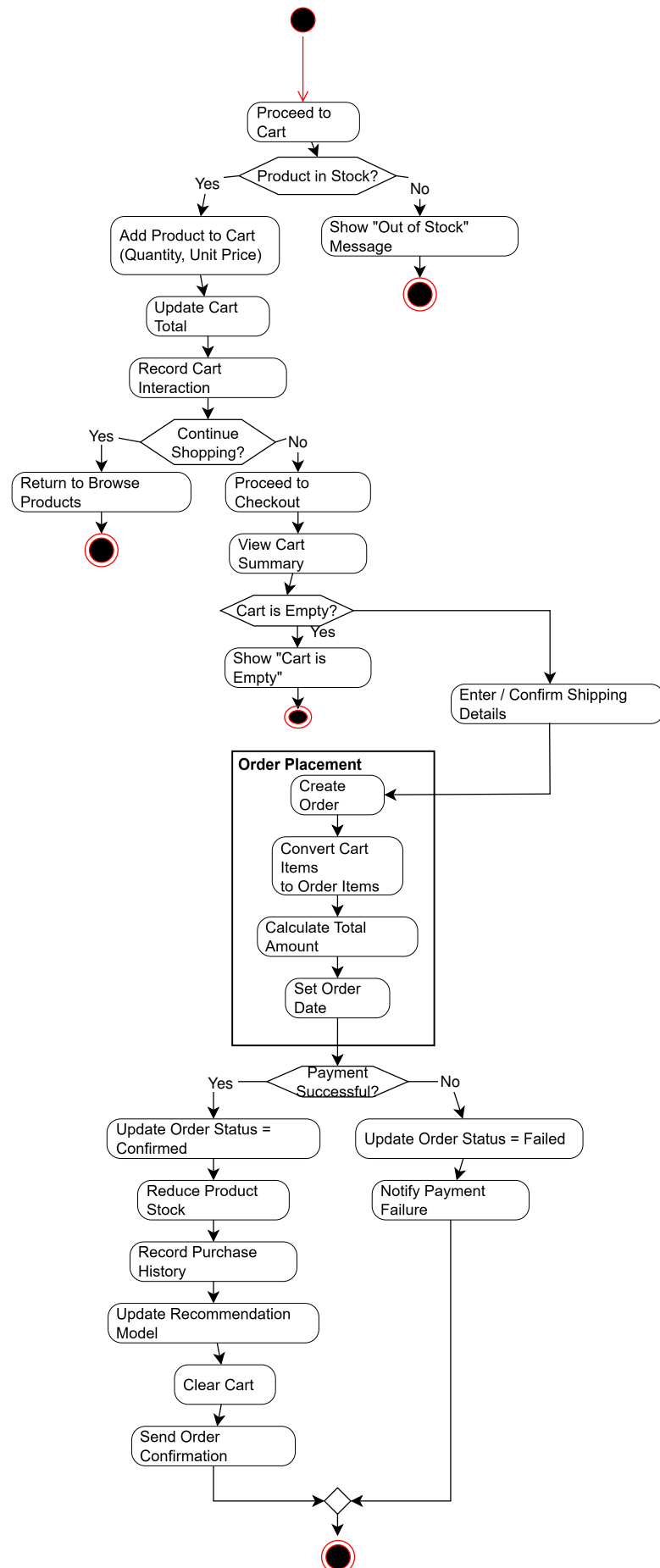


Figure 3.6: Activity Diagram - Cart & Order placement & Payment

CHAPTER 4:

METHODOLOGY

4.1 Basic System Architecture

The system follows a three-tier client-server architecture. The presentation tier is a React.js SPA communicating with the backend through RESTful HTTP APIs. The application tier, built with Django REST Framework, handles business logic, authentication, recommendation computation, and third-party API integration. The data tier is a MySQL database accessed through Django's ORM.

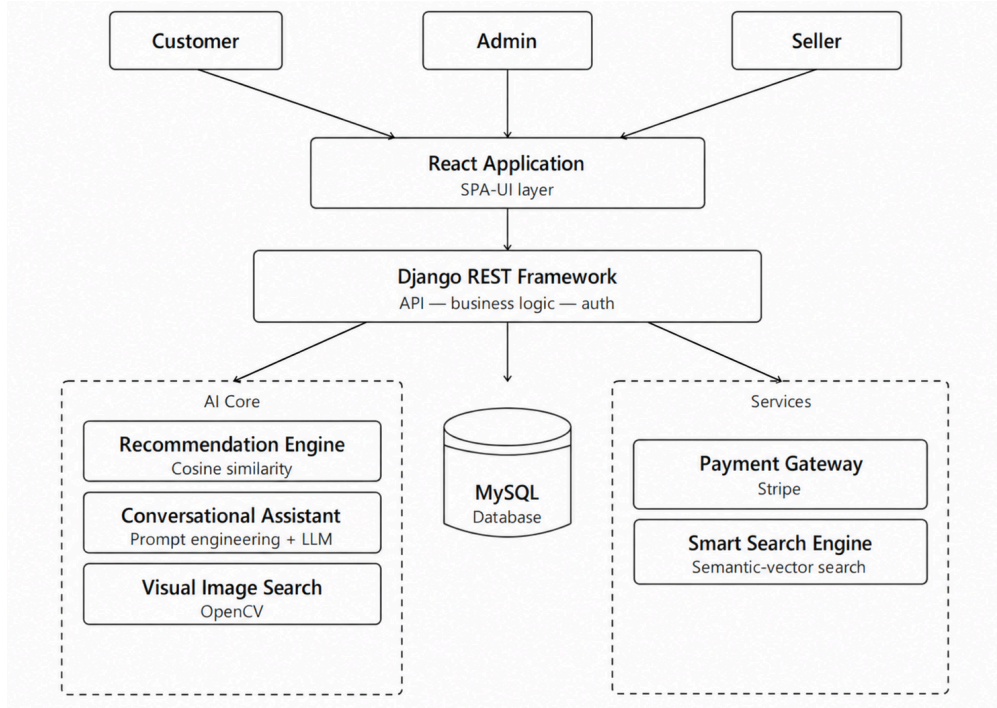


Figure 4.1: System Architecture Diagram

The AI Core, the Recommendation Engine and Semantic Search Engine, sits in the application tier as service modules, with a Visual Search Engine planned for the remaining phase. The Recommendation Engine computes weighted cosine similarity over product attribute vectors using Scikit-learn, invoked by the DRF views serving the product detail and homepage endpoints. External services, Khalti for payments and an email provider for notifications, interface with the application tier over HTTPS.

4.1.1 System Flowcharts

Recommendation Engine Flow

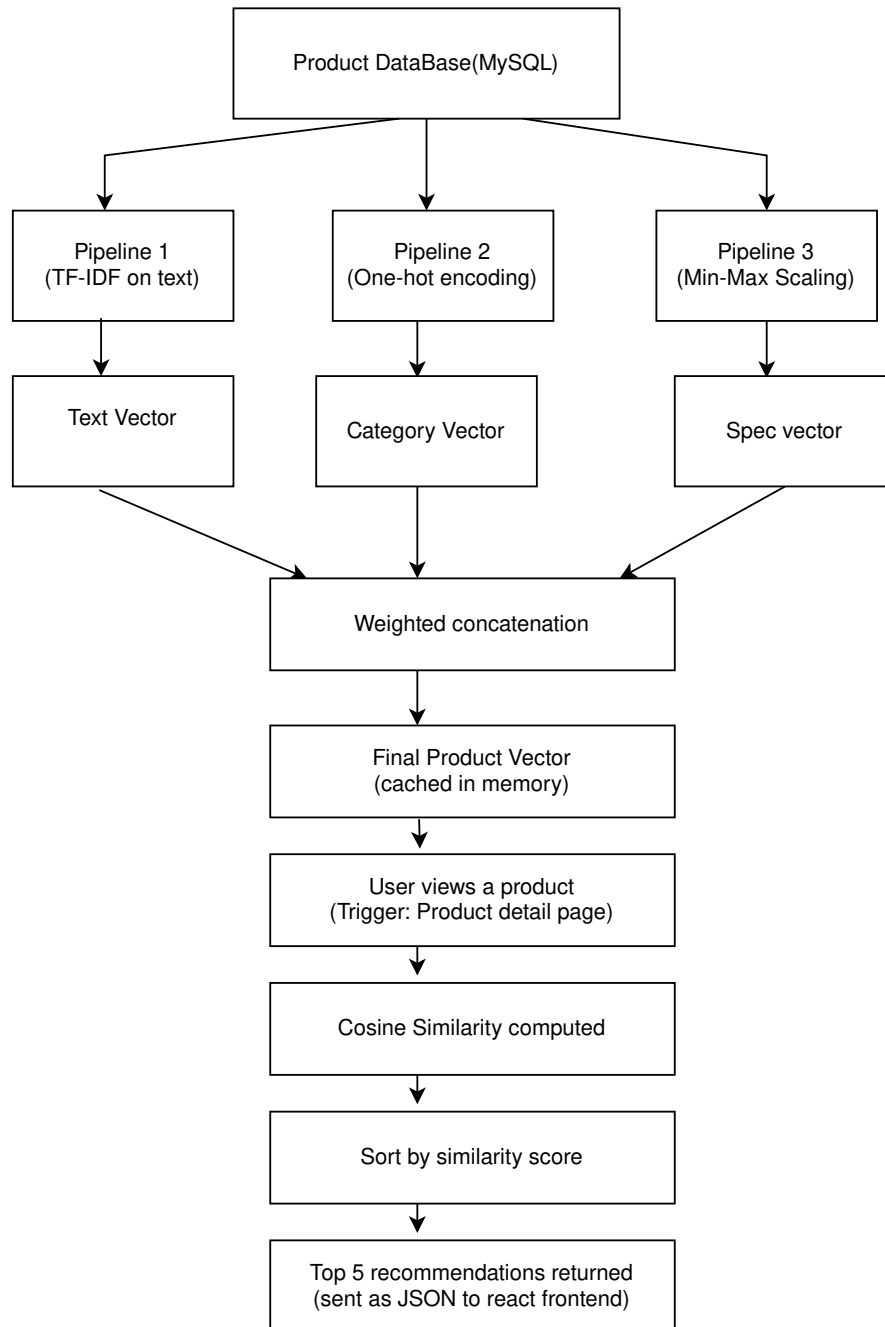


Figure 4.2: Recommendation Engine Flow Diagram

The engine begins when a user views, searches, or purchases a product. Category, brand, specifications, and description are pulled from the catalog; features are extracted using TF-IDF (text), one-hot encoding (categorical), and min-max scaling (numerical), then combined via weighted concatenation into a single product vector. Cosine similarity

is computed between products, and the top-ranked results are shown as personalized recommendations.

4.1.2 Algorithm: Cosine Similarity for Product Recommendation

Purpose and Application

Cosine similarity measures how alike two products are based on their attribute vectors, ranking the catalog by decreasing similarity to the product a user is currently viewing. The engine retrieves the viewed product’s vector, computes similarity against every other product, and returns the top-5 most similar products on the product detail page and homepage.

Why Cosine Similarity

Three reasons drove this choice. **Cold-start robustness:** unlike collaborative filtering, it requires no prior user history, making it suitable for a newly launched platform. **Efficiency:** pairwise similarity over TF-IDF, one-hot, and numerical matrices via Scikit-learn needs no GPU or training pipeline. **Interpretability:** the score directly reflects similarity in category, brand, price, and specifications, making recommendations auditable.

Feature Vector Construction

Each product is represented as $\mathbf{v} \in \mathbb{R}^n$, built from three attribute blocks combined by weighted concatenation.

Text block (TF-IDF). For term t and document d from a catalog of N descriptions:

$$\text{tfidf}(t, d) = tf(t, d) \times \log\left(\frac{N}{df(t)}\right) \quad (4.1)$$

where $tf(t, d)$ is the frequency of t in d , and $df(t)$ is the number of descriptions containing t . Equation (4.1) down-weights common words (e.g. “smartphone”) and up-weights distinctive terms, giving the text vector $\mathbf{t}_i$.

Categorical block (one-hot encoding). Category, sub-category, brand, and OS become a binary indicator vector $\mathbf{c}_i$.

Specification block (min-max scaling). Eight numerical fields (price, RAM, storage,

display size, refresh rate, battery capacity, weight, rating) are rescaled to $[0, 1]$ so no single field dominates by scale:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (4.2)$$

giving the numeric vector $\mathbf{s}_i$.

Weighted concatenation.

$$\mathbf{v}_i = \left[w_{\text{text}} \cdot \mathbf{t}_i \parallel w_{\text{cat}} \cdot \mathbf{c}_i \parallel w_{\text{spec}} \cdot \mathbf{s}_i \right] \quad (4.3)$$

where $\parallel$ is concatenation and w_{text} , w_{cat} , w_{spec} are the block weights from Section 4.1.4.

Given vectors $\mathbf{A}$ and $\mathbf{B}$:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \quad (4.4)$$

with a score in $[0, 1]$: 1 means identical attribute space, 0 means no shared attributes.

Algorithm 1 Content-Based Product Recommendation using Cosine Similarity**Require:** Product catalog $\mathcal{P} = \{p_1, p_2, \dots, p_m\}$, viewed product p_q **Ensure:** Ordered list of top- N recommended products**Step 1: Feature Vector Construction**for each product $p_i \in \mathcal{P}$ do

Encode category, sub-category, brand, and OS as one-hot vectors

 Normalize price and specification fields to $[0, 1]$ using min-max scaling

Compute TF-IDF vector from product description

 Concatenate weighted features: $\mathbf{v}_i \leftarrow [w_{\text{text}} \cdot \text{TF-IDF} \parallel w_{\text{cat}} \cdot \text{category} \parallel w_{\text{spec}} \cdot \text{specs}]$

end for

Step 2: Similarity ComputationRetrieve feature vector $\mathbf{v}_q$ for the viewed product p_q for each product $p_i \in \mathcal{P}, p_i \neq p_q$ do

$$s_i \leftarrow \frac{\mathbf{v}_q \cdot \mathbf{v}_i}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}_i\|}$$

end for

Step 3: Ranking and SelectionSort all products by s_i in descending orderReturn top- N products (default $N = 5$) as recommendations**Weight Selection and Validation**

The weights are chosen by a grid search scored against a relevance rule: two products are relevant if they share a category and satisfy at least two of three conditions, comparable price, comparable specifications, and overlapping description keywords. This majority-vote rule replaces a simpler same-category-only rule, since the category vector is itself one of the weighted inputs; scoring purely against category membership would reward the search for leaning on that vector regardless of actual recommendation quality (“label circularity”).

For a candidate weight triple, vectors are built with Equation (4.3) over a sampled subset of the catalog, pairwise similarities computed with Equation (4.4), and the candidate scored by the Pearson correlation between similarity values and the binary relevance

labels y :

$$\text{score}(w_{\text{text}}, w_{\text{cat}}, w_{\text{spec}}) = \frac{\text{Cov}(\text{sim}, y)}{\sigma_{\text{sim}} \sigma_y} \quad (4.5)$$

The search selects the weight triple maximizing Equation (4.5), normalized so the weights sum to one:

$$w'_i = \frac{w_i}{w_{\text{text}} + w_{\text{cat}} + w_{\text{spec}}}, \quad i \in \{\text{text}, \text{cat}, \text{spec}\} \quad (4.6)$$

To confirm the result was not a fragile, one-off outcome, a sensitivity sweep varied each weight across a range of values while holding the ratio of the other two fixed, re-scoring recommendation quality with Equation (4.5) at each point. This sweep and its results are presented in Section 5.1.2.

Evaluation Metrics

The tuned engine is benchmarked against a random-recommendation baseline using five standard ranking metrics, computed against the same majority-vote relevance labels used in Equation (4.5). For a query product q with a top- K list and relevant-item set R_q :

$$\text{Precision@}K = \frac{|\{\text{top-}K \text{ recommendations for } q\} \cap R_q|}{K} \quad (4.7)$$

$$\text{Recall@}K = \frac{|\{\text{top-}K \text{ recommendations for } q\} \cap R_q|}{|R_q|} \quad (4.8)$$

Because $|R_q|$ often exceeds K , no recommender can reach a perfect Recall@ K ; the theoretical ceiling is:

$$\text{Recall@}K_{\max} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{\min(K, |R_q|)}{|R_q|} \quad (4.9)$$

$$\text{HitRate@}K = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \mathbb{1}_{\left[|\{\text{top-}K \text{ recommendations for } q\} \cap R_q| > 0\right]} \quad (4.10)$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q} \quad (4.11)$$

$$\text{NDCG@K} = \frac{\sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}}{\sum_{i=1}^{|R_q|} \frac{1}{\log_2(i+1)}} \quad (4.12)$$

Precision@K measures the fraction of top- K recommendations that are relevant; Recall@K the fraction of relevant items captured; HitRate@K whether at least one hit occurred; MRR how close the first relevant item sits to rank 1; and NDCG@K rewards both including and correctly ranking relevant items, normalized to $[0, 1]$.

Implementation

The feature matrix is pre-computed over the full catalog at server startup and cached in memory using Scikit-learn's `TfidfVectorizer`, `OneHotEncoder`, `MinMaxScaler`, and `cosine_similarity`. On a recommendation request, the system looks up the pre-computed similarity row, sorts scores, filters out-of-stock items, and returns the top- N product IDs, which are used to query MySQL for full product details serialized back to the frontend.

4.2 Agile SCRUM Model

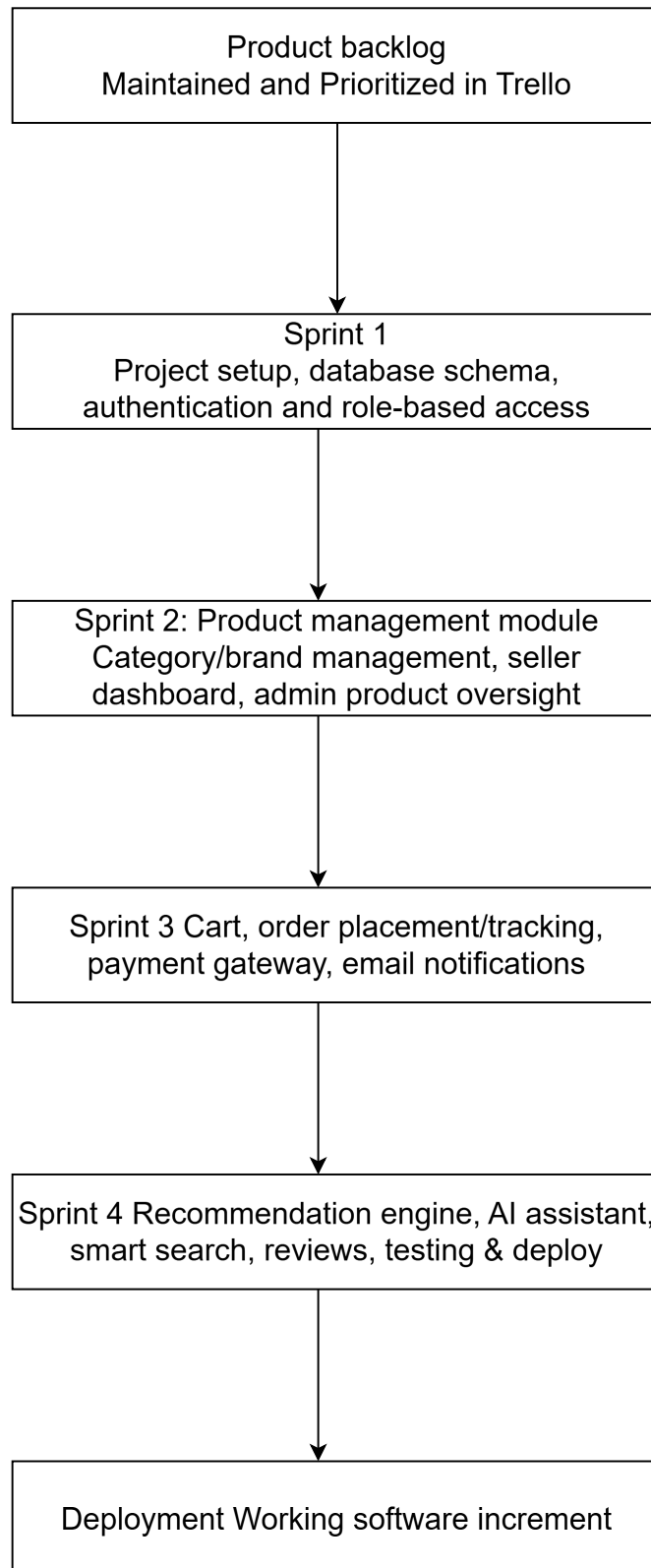


Figure 4.3: Agile Model Diagram

Development follows Agile SCRUM across four two-week sprints:

- **Sprint 1:** Setup, database schema, authentication (registration, login, JWT), role-based access control.
- **Sprint 2:** Product management, category/brand management, seller dashboard, admin product oversight.
- **Sprint 3:** Cart, order placement/tracking, payment gateway, email notifications.
- **Sprint 4:** Recommendation engine validation, OpenCV visual search, smart search with price filtering, review system, final testing and deployment.

4.3 Dataset Description

The recommendation engine uses an electronics dataset of approximately 10,000 entries (roughly 6,000 smartphones, 4,000 laptops), each with a product identifier, name, category, sub-category, brand, OS, free-text description, and numerical fields: price (NPR), RAM (GB), storage (GB), display size (inches), refresh rate (Hz), battery capacity (mAh), weight (grams), and rating.

Rows with missing values are dropped before feature extraction. Descriptions are vectorized with TF-IDF; category, sub-category, brand, and OS with one-hot encoding; numeric fields with min-max scaling. Because a full pairwise search across all weight combinations is expensive at 10,000 rows, the weight search and its evaluation (Chapter 5) run on a stratified sample of a few hundred products, with the resulting weights applied to the full catalog. During deployment, this initial dataset will be gradually supplemented by real listings from verified sellers.

4.4 Tools Used

Category	Technology	Purpose
Frontend	React.js, Tailwind CSS	UI development
Backend	Django, Django REST Framework	API and business logic
Database	MySQL	Relational data storage
AI / ML	Scikit-learn, OpenCV	Recommendation engine, visual image search
Authentication	JSON Web Tokens (JWT)	Secure session management
Payment	Khalti	Transaction processing
Version Control	Git, GitHub	Source code management
Project Management	Trello	SCRUM backlog and sprint tracking
IDE	VS Code	Development environment
API Testing	Postman	Endpoint testing and documentation

Table 4.1: Development Tools and Technologies

CHAPTER 5:

RESULTS AND ANALYSIS

5.1 Work Completed

5.1.1 Backend Development (Django REST Framework)

- Set up the Django project with MySQL integration, using an `.env` file to keep credentials and secrets out of source control.
- Designed the schema across the `catalog` and `accounts` apps, matching the ER diagram: User, Seller, Product, Brand, Category.
- Built a custom User model with a `role` field (Customer, Seller, Admin), set up before the first migration to avoid switching User models later.
- Implemented JWT authentication via `djangorestframework-simplejwt`: registration, login, and token validation on protected endpoints.
- Implemented role-based permission classes (`IsSeller`, `IsAdmin`) restricting access by role.
- Built the seller onboarding workflow: a seller registers, enters a pending state, and is admin-approved before listing products, enforced at the API level.
- Built seller-side product management: add a new product, or claim an unclaimed catalog listing under a custom price and stock quantity.
- Developed catalog APIs: category/brand listing, keyword search with price-range filtering, and a featured-products endpoint balancing phones and laptops.
- Set up media and static file handling for product images.
- Resolved a MySQL connection failure caused by special characters in the password (URL-encoding) and a migration conflict from a duplicate model definition.

5.1.2 Recommendation Engine Development and Evaluation

- Built the core pipeline: TF-IDF, one-hot encoding, and min-max scaling combined via weighted concatenation and scored with cosine similarity, precomputed once at startup.
- Identified and fixed a circularity issue in the initial weight search (a same-category-only relevance proxy), replacing it with the majority-vote relevance rule described in Section 4.1.4.
- Under the corrected rule, the grid search selected $w_{\text{text}} = 0.692$, $w_{\text{cat}} = 0.231$, $w_{\text{spec}} = 0.077$, text-dominant, since numeric specs alone already separate laptops from phones almost as cleanly as category, adding little once category is accounted for.
- An independent sensitivity sweep (each weight varied 0.1–0.9, others held at baseline ratio) confirmed the result was not a fluke: text peaked near its selected value, category stayed flat, and spec declined monotonically, all consistent with the grid search’s choice.

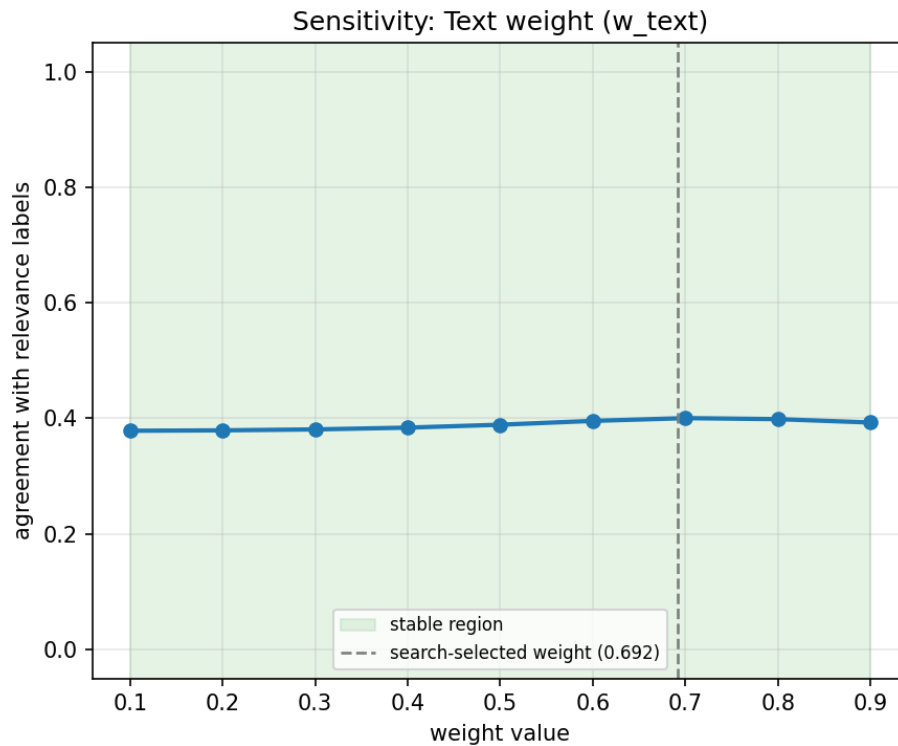


Figure 5.1: Weight Sensitivity Analysis – Text Weight

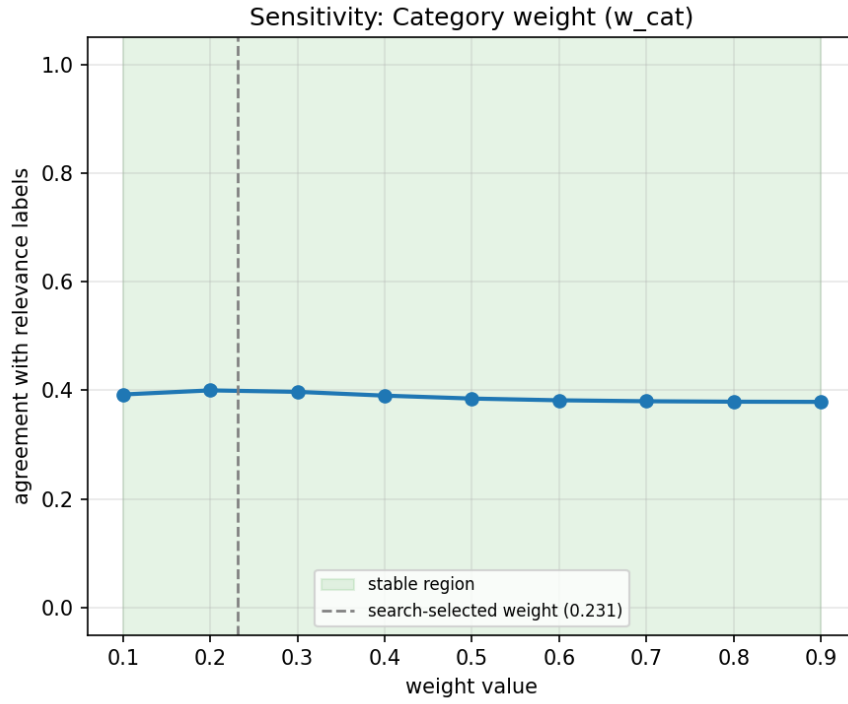


Figure 5.2: Weight Sensitivity Analysis – Category Weight

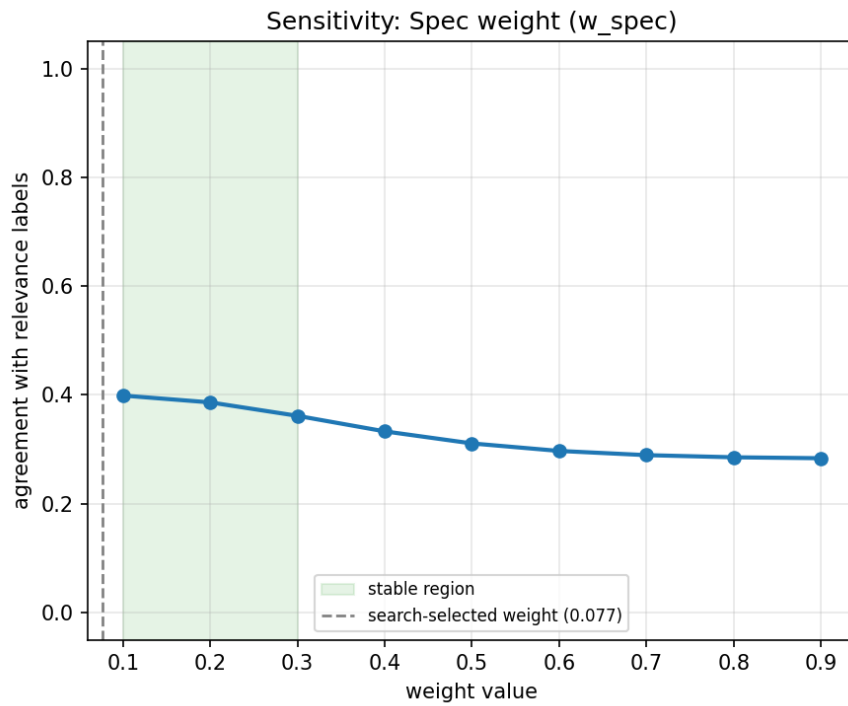


Figure 5.3: Weight Sensitivity Analysis – Spec Weight

- Benchmarked against a random-recommendation baseline on the same relevance rule: Precision@5 0.645 vs 0.057, Hit Rate@5 0.950 vs 0.251, MRR 0.864 vs

0.170, NDCG@5 0.682 vs 0.057.

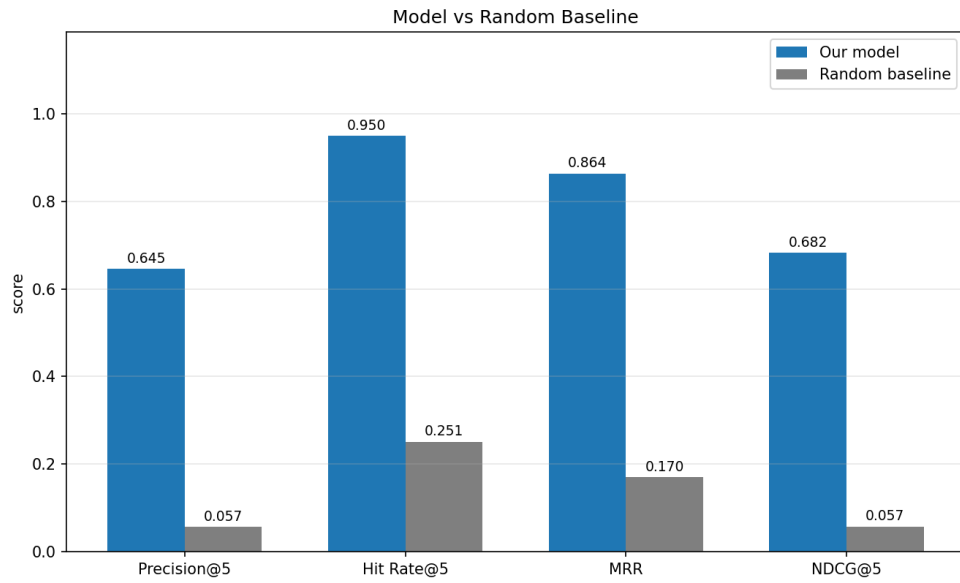


Figure 5.4: Recommendation Quality: Model vs Random Baseline

- Raw Recall@5 was initially low (0.125). This traced to the relevant-item pool being large relative to $k = 5$: Equation (4.9) gives a theoretical ceiling of ~ 0.247 for this dataset, so the engine reaches $\sim 51\%$ of the maximum achievable.

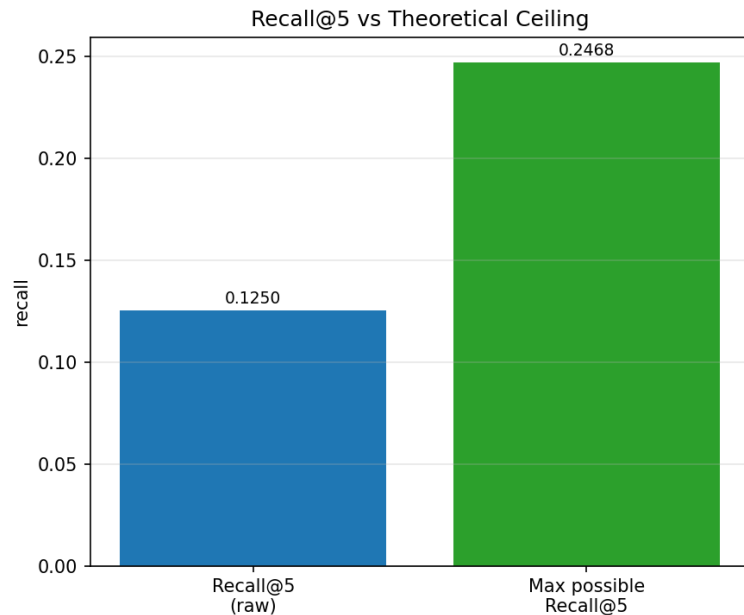


Figure 5.5: Recall@5 versus its Theoretical Ceiling

- Ran an end-to-end demonstration (5 laptops + 5 smartphones, top-5 each) on the

full 10,000-row catalog, confirming the pipeline works outside the smaller tuning sample (terminal output in Appendix D).

5.1.3 Frontend Development (React with Tailwind CSS)

- Built a combined login/registration page with a role selector (Customer or Seller), connected to the JWT endpoints.
- Built the customer onboarding preference page (category interest, budget range, priority specification), saved to the account.
- Built the customer homepage, fetching personalized recommendations from saved preferences.
- Built the product detail page: specifications, description, pricing, stock, and a similar-products section powered by the recommendation engine.
- Built the seller onboarding page for business name and contact information.
- Built the seller dashboard: add-product form, search-and-claim interface, live view of listed products, and a verification status banner.

5.1.4 Version Control and Team Collaboration

- Set up Git with the team's shared GitHub repository.
- Excluded a large, regenerable similarity-matrix artifact from version control via `.gitignore` rather than forcing it through.
- Pushed the complete backend and frontend implementation to a shared feature branch for review.

5.1.5 Verification

- Every API endpoint was manually tested in Postman before frontend integration, checking request/response formats and status codes.
- Role-based access was verified with and without valid JWTs, and across roles, confirming unauthorized requests return 401/403 rather than succeeding.

- Database migrations were checked against the live MySQL schema after every model change, especially after the earlier migration conflict.
- The recommendation engine’s relevance definition was verified for internal consistency, confirming the weight-search and evaluation scripts share one relevance function rather than each defining its own.
- The engine’s output was checked to correctly exclude out-of-stock items and the viewed product itself.

5.1.6 Validation

- The full customer journey, registration through product detail viewing, was validated end-to-end.
- The full seller journey, registration, pending state, admin approval, listing, was validated end-to-end.
- The personalized homepage was validated across category, budget, and priority-spec combinations, catching and fixing an edge case where selecting “Both” categories returned only one product type.
- The tuned weights were validated independently of the grid search itself, through the sensitivity analysis and random-baseline comparison in Section 5.1.2.
- Product images were validated across the seller dashboard, homepage, and detail page after a media path mismatch was found and corrected.
- Multiple test accounts across both roles were used to repeatedly walk through every implemented flow before code was pushed.

5.2 Work Remaining

- Shopping cart, order placement, and tracking, deferred to prioritize the recommendation and personalization features first.
- Khalti payment gateway integration.
- OpenCV-based visual image search.

- A custom administrative dashboard for orders, payments, coupons, and analytics; Django's built-in admin currently handles these functions.
- Semantic and synonym-aware search (e.g. “thin laptop” understood as low-weight devices).
- Replacing the majority-vote relevance rule with a small independent human-judgment validation sample, if time permits.
- Final visual design refinement, handled by other team members.
- Full integration testing across all modules ahead of final submission.

CHAPTER 6:

CONCLUSION

This mid-term progress report has presented the design, architecture, and current implementation status of the Intelligent Adaptive Multi-Vendor Electronics Marketplace. The project has progressed from requirement analysis and system design through to a functional backend and frontend foundation, validating the proposed architecture.

The team has implemented JWT-based authentication and role-based access control across customer, seller, and administrator roles; a seller onboarding workflow with administrative approval; a category- and brand-organized product catalog; and a content-based recommendation engine using weighted cosine similarity over TF-IDF, one-hot, and min-max scaled features. The engine's weight-selection methodology was corrected after an internal circularity issue was identified, its weights were independently validated through a sensitivity analysis, and its quality was benchmarked against a random baseline. The corresponding frontend, authentication, onboarding, seller dashboard, personalized homepage, and product detail pages, has been integrated with the backend through a RESTful API layer, with MySQL, CORS, and version control established for ongoing development.

These completed components show the architecture is sound and that the content-based approach addresses the cold-start problem inherent to a newly launched marketplace, while standing up to independent scrutiny of its evaluation methodology.

Several modules remain: the cart and checkout system, order processing, Khalti integration, and OpenCV-based visual search. Semantic search, letting customers describe requirements in natural language, is also planned for the remaining phase.

The remaining work is scheduled across the upcoming sprints and is expected to bring the platform to a fully functional state spanning the complete purchasing journey, from discovery through fulfillment, demonstrating a practical, cost-effective approach to embedding intelligent, personalized features into a multi-vendor platform for the Nepali electronics retail market.

REFERENCES

- [1] E. Turban, D. King, J. K. Lee, T.-P. Liang, and D. C. Turban, *Electronic Commerce: A Managerial and Social Networks Perspective*, 8th ed. New York, NY, USA: Springer, 2015.
- [2] K. C. Laudon and C. G. Traver, *E-Commerce: Business, Technology, Society*, 17th ed. Hoboken, NJ, USA: Pearson, 2021.
- [3] L. Richardson, *RESTful Web APIs: Services for a Changing World*. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [4] M. Masse, *REST API Design Rulebook*. Sebastopol, CA, USA: O'Reilly Media, 2011.
- [5] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, Jan.–Feb. 2003.
- [6] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.
- [7] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, "Recommender systems survey," *Knowl.-Based Syst.*, vol. 46, pp. 109–132, Jul. 2013.
- [8] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *ACM Comput. Surv.*, vol. 52, no. 1, pp. 1–38, Jan. 2019.
- [9] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval: The Concepts and Technology behind Search*, 2nd ed. New York, NY, USA: ACM Press/Addison-Wesley, 2011.
- [10] K.-P. Yee, K. Swearingen, K. Li, and M. Hearst, "Faceted metadata for image search and browsing," in *Proc. SIGCHI Conf. Human Factors Comput. Syst. (CHI)*, 2003, pp. 401–408.

- [11] P. Morville and L. Rosenfeld, *Information Architecture for the World Wide Web*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2006.
- [12] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA, USA: O'Reilly Media, 2008.

APPENDICES

6.1 APPENDIX A: Project Timeline

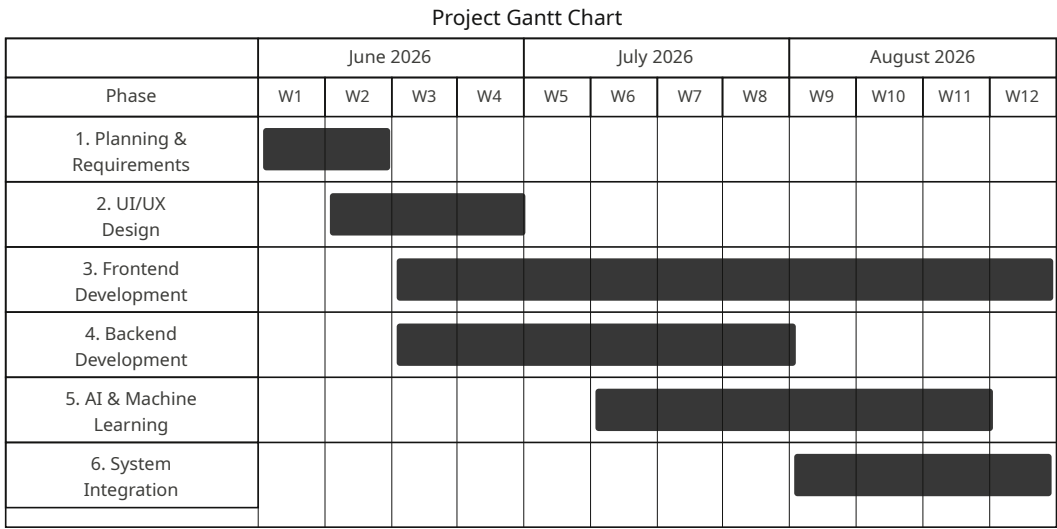


Figure 6.1: Project Timeline Gantt Chart

The project runs over a 16-week period from June to September 2026: requirements and design (weeks 1–2), Sprint 1 (weeks 3–4), Sprint 2 (weeks 5–8), Sprint 3 (weeks 9–12), Sprint 4 and remaining AI feature integration (weeks 13–15), final testing and submission (week 16).

6.2 APPENDIX B: Technical Specifications

6.2.1 Software Requirements

Category	Technology
Frontend	React.js, Tailwind CSS
Backend	Django, Django REST Framework
Database	MySQL
AI / ML	Scikit-learn, OpenCV
Authentication	JSON Web Token (JWT)
Payment Gateway	Khalti
Version Control	Git, GitHub
IDE & Testing	Visual Studio Code, Postman
Project Management	Trello

Table 6.1: Software Requirements of the Proposed System

6.2.2 Hardware Requirements

- Personal computer or laptop
- Intel Core i5 / AMD Ryzen 5 processor or higher
- Minimum 8 GB RAM
- Minimum 256 GB SSD storage
- Stable internet connection (minimum 10 Mbps recommended)

6.3 APPENDIX C: Recommendation Engine Terminal Output

```
{
  "viewed_product": {
    "product_id": "PHN-06471",
    "product_name": "Pixel 8a 8GB/1024GB - Silver",
    "category": "Smartphone"
  },
  "recommendations": [
    {
      "id": "PHN-05544",
      "name": "Pixel 8 1024GB - Phantom Black",
      "category": "Smartphone",
      "similarity_score": 0.9943
    },
    {
      "id": "PHN-00700",
      "name": "Pixel 7a 1024GB - Cream",
      "category": "Smartphone",
      "similarity_score": 0.9928
    },
    {
      "id": "PHN-04723",
      "name": "Pixel 8 1024GB - Purple",
      "category": "Smartphone",
      "similarity_score": 0.9925
    },
    {
      "id": "PHN-09724",
      "name": "Pixel 7 Pro 1024GB - Red",
      "category": "Smartphone",
      "similarity_score": 0.9911
    },
    {
      "id": "PHN-01204",
      "name": "Pixel 7 Pro 1024GB - Black",
      "category": "Smartphone",
      "similarity_score": 0.9905
    }
  ]
}
```

Figure 6.2: Terminal Output: Top-5 Recommendation Demonstration

```
{
  "viewed_product": {
    "product_id": "PHN-01488",
    "product_name": "Realme Narzo 60 12GB/256GB - Green",
    "category": "Smartphone"
  },
  "recommendations": [
    {
      "id": "PHN-07890",
      "name": "Realme C67 256GB - Cream",
      "category": "Smartphone",
      "similarity_score": 0.9946
    },
    {
      "id": "PHN-08424",
      "name": "Realme C67 128GB - Purple",
      "category": "Smartphone",
      "similarity_score": 0.9935
    },
    {
      "id": "PHN-03010",
      "name": "Realme GT Neo5 512GB - Silver",
      "category": "Smartphone",
      "similarity_score": 0.991
    },
    {
      "id": "PHN-08785",
      "name": "Realme 12 Pro+ 128GB - Titanium Blue",
      "category": "Smartphone",
      "similarity_score": 0.9903
    },
    {
      "id": "PHN-03278",
      "name": "Realme C67 16GB/128GB - Silver",
      "category": "Smartphone",
      "similarity_score": 0.9893
    }
  ]
}
```

Figure 6.3: Terminal Output: Top-5 Recommendation Demonstration

6.4 APPENDIX D: User Interface Screenshots

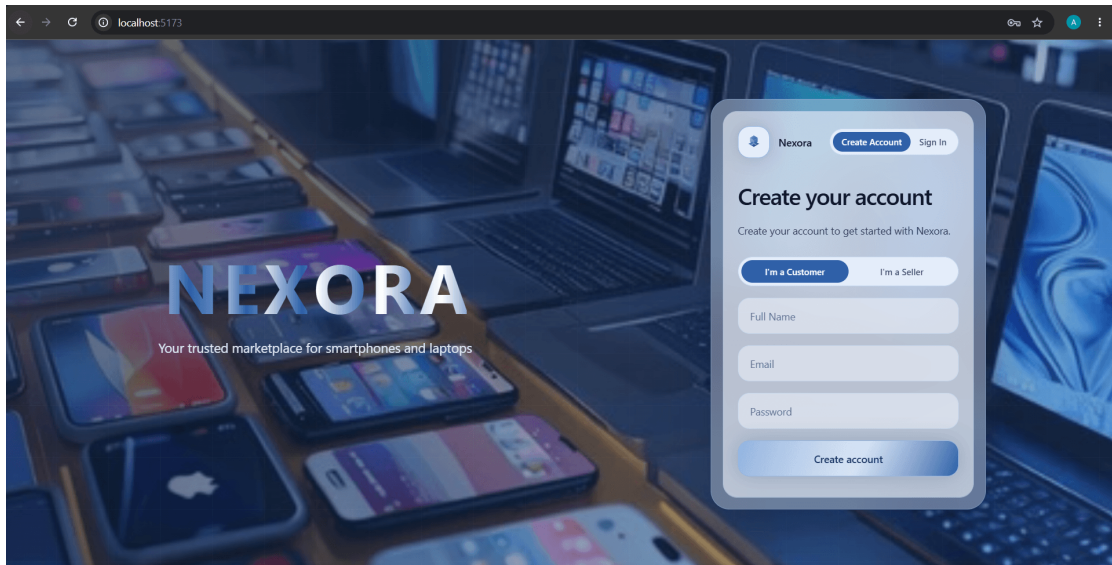


Figure 6.4: Registration Interface

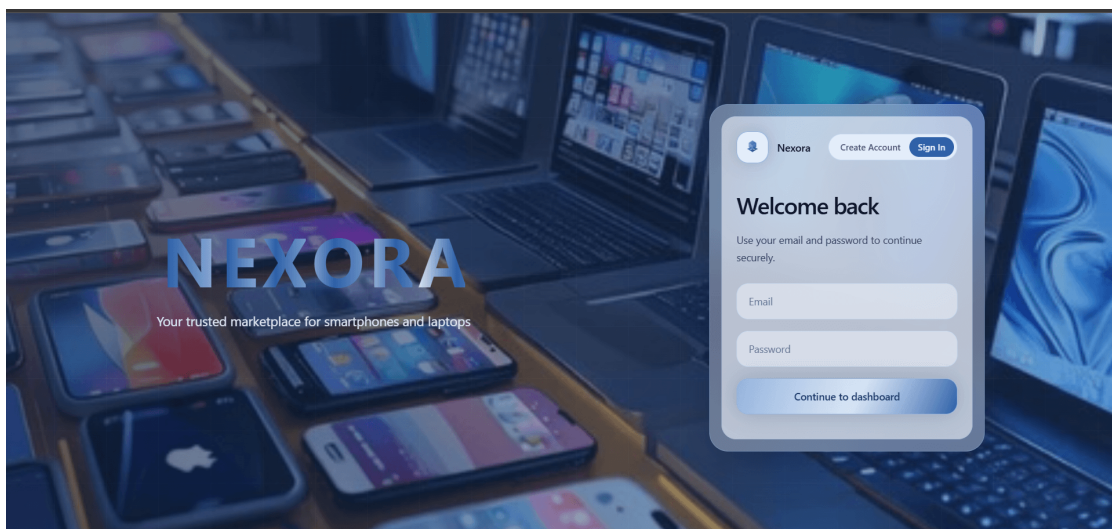
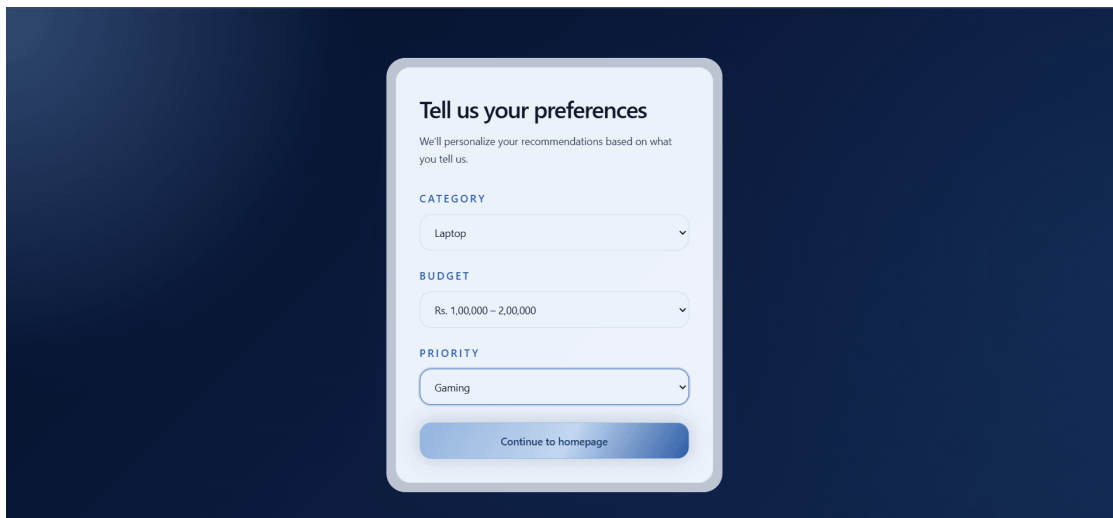


Figure 6.5: Sign In Interface



Tell us your preferences

We'll personalize your recommendations based on what you tell us.

CATEGORY

Laptop

BUDGET

Rs. 1,00,000 – 2,00,000

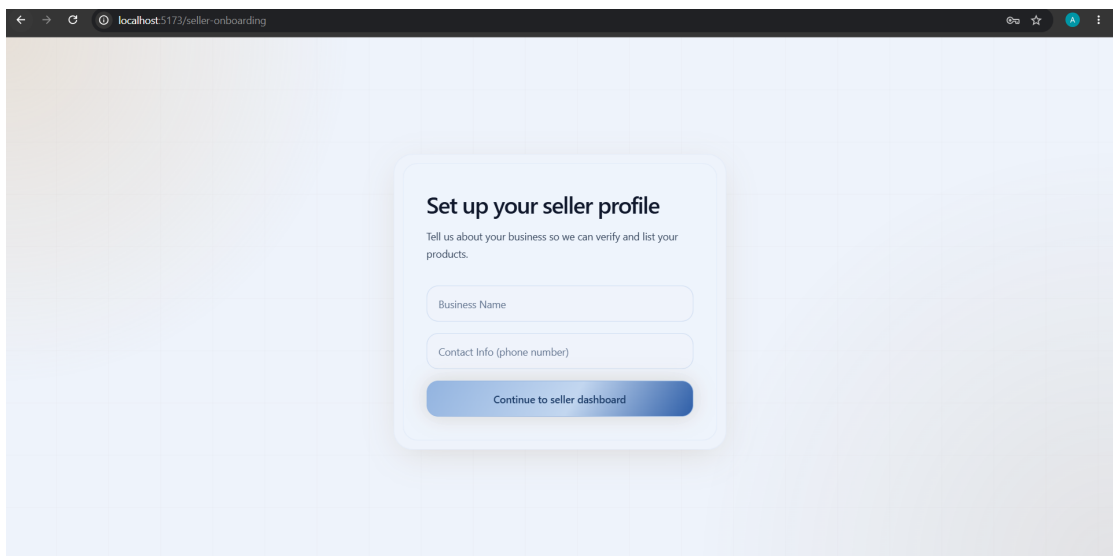
PRIORITY

Gaming

Continue to homepage

The interface is a dark blue modal box with a white background for the form. It contains three dropdown menus for 'CATEGORY' (Laptop), 'BUDGET' (Rs. 1,00,000 – 2,00,000), and 'PRIORITY' (Gaming). A blue button at the bottom says 'Continue to homepage'.

Figure 6.6: User Preference Selection Interface



Set up your seller profile

Tell us about your business so we can verify and list your products.

Business Name

Contact Info (phone number)

Continue to seller dashboard

The interface is a light blue modal box with a white background for the form. It contains two text input fields for 'Business Name' and 'Contact Info (phone number)'. A blue button at the bottom says 'Continue to seller dashboard'.

Figure 6.7: Seller Onboarding Interface

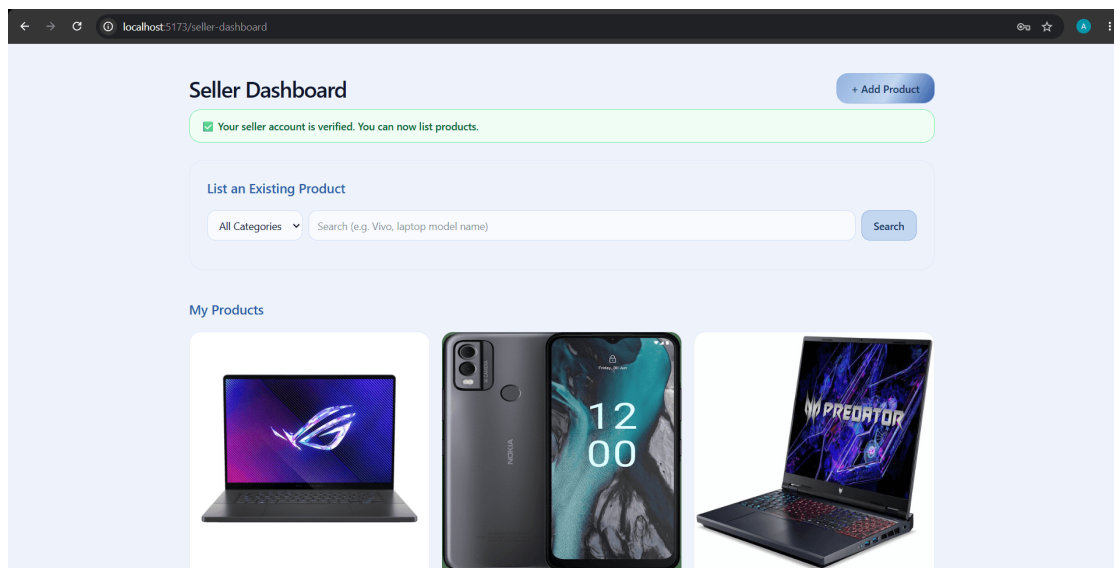


Figure 6.8: Seller Dashboard Interface

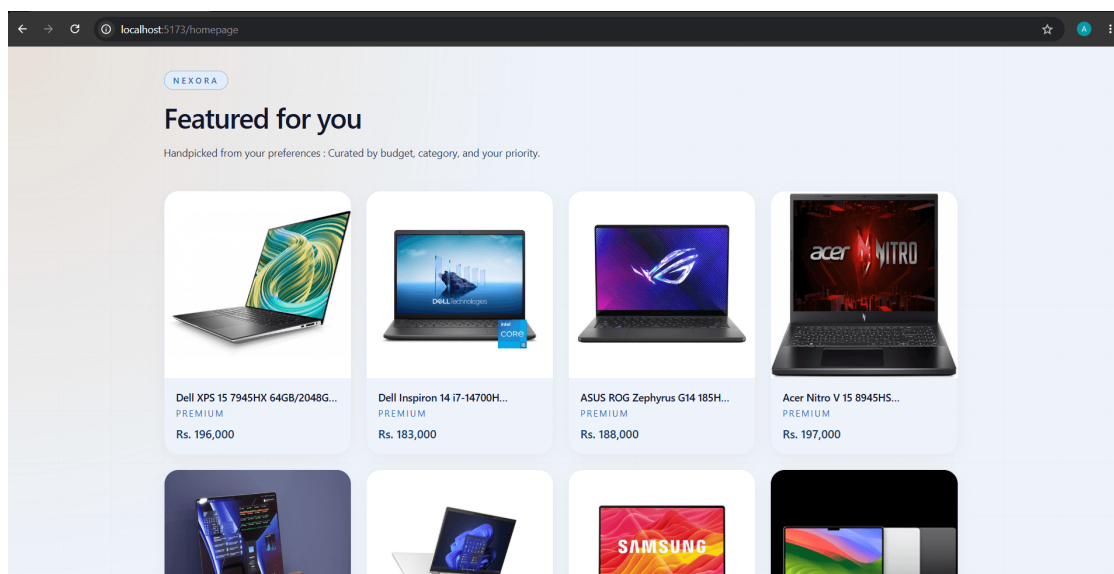


Figure 6.9: Featured Products Interface

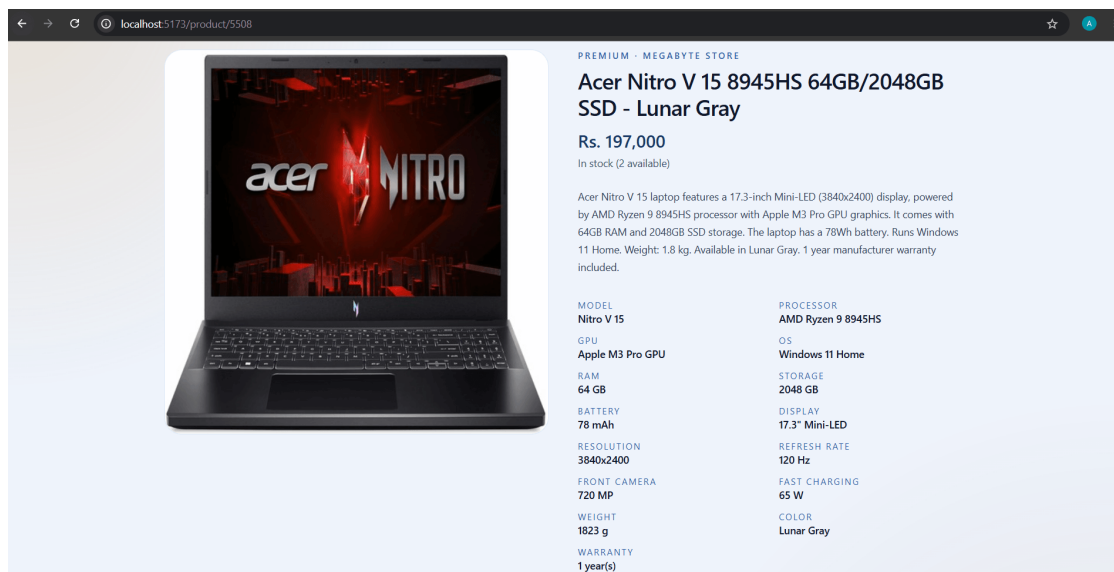


Figure 6.10: Product Detail Interface